Apache Spark

Guia Técnico Completo

Instalação, Configuração e Arquitetura para Processamento Distribuído de Dados

Documentação Técnica

Professor Benir Falcão Pistille

Março 2026

Versão 1.0

Sumario

Clique com o botão direito e selecione 'Atualizar Campo' para atualizar os números de pagina

Sumario **2**

1. Introducao ao Apache Spark **5**

1.1 O que e Apache Spark 5

Principais Caracteristicas 5

1.2 Historia e Evolucao 5

Linha do Tempo 6

1.3 Casos de Uso 6

Processamento de Dados em Larga Escala (ETL) 6

Analise de Dados em Tempo Real 6

Machine Learning 7

Processamento de Grafos 7

2. Arquitetura do Apache Spark **7**

2.1 Componentes Principais 7

Spark Core 7

Spark SQL 8

Spark Streaming 8

MLlib 8

GraphX 8

2.2 Modelo de Execucao 8

Driver Program 9

Cluster Manager 9

Worker Nodes e Executors 9

3. Instalacao e Configuracao **10**

3.1 Pre-requisitos 10

Requisitos de Hardware 10

Requisitos de Software 10

3.2 Instalacao Passo a Passo 11

Passo 1: Instalar Java 11
Passo 2: Baixar o Apache Spark 11
Passo 3: Configurar Variaveis de Ambiente 12
Passo 4: Verificar Instalacao 12
3.3 Configuracao do Cluster 12
Modo Standalone 12
Configuracao spark-defaults.conf 13
Configuracao do Ambiente 13
4. Processamento ETL 14
4.1 Extracao de Dados 14
4.2 Transformacao de Dados 15
4.3 Carga de Dados 16
4.4 Otimizacao de Performance 17
5. Machine Learning com MLlib 18
5.1 Tipos de Algoritmos 18
Classificacao 18
Regressao ..... **Erro! Indicador não definido.**
Clustering 19
Filtragem Colaborativa 19
5.2 Exemplo de Pipeline ML 19
5.3 Feature Engineering 20
6. Analise de Risco 21
6.1 Fontes de Dados de Risco 22
6.2 Modelos de Risco 22
Score de Credito 22
Deteccao de Fraude 23
6.3 Processamento em Tempo Real 24
6.4 Governanca e Compliance 25
7. Tecnologias Emergentes 26
7.1 Lakehouse Architecture 26
7.2 Kubernetes e Cloud Native 27

7.3 AI e Deep Learning 27
7.4 Streaming Avancado 28
7.5 Observabilidade 28

8. Conclusao 29

8.1 Resumo dos Beneficios 29
8.2 Proximos Passos 29
8.3 Recursos Adicionais 29

1. Introdução ao Apache Spark

1.1 O que e Apache Spark

Apache Spark e um framework de processamento de dados distribuído de código aberto, projetado para executar operações de processamento de dados em larga escala de forma rápida e eficiente. Desenvolvido originalmente na UC Berkeley em 2009, o Spark tornou-se um dos projetos mais ativos da Apache Software Foundation.

O Spark fornece uma interface para programação de clusters com paralelismo de dados implícito e tolerância a falhas. Ele e projetado para lidar com cargas de trabalho de processamento batch e stream processing, oferecendo APIs em Java, Scala, Python e R.

Objetivos do Projeto Entregas:

- Introdução sobre a solução e Benefícios
- Objetivo do Projeto
- Descrição da plataforma
- BPV (Business Process View) automação de negócios e diagrama
- Fluxograma lógico de Instalação e Configuração
- Referências e Pesquisa

Principais Características

- Processamento em memória: até 100x mais rápido que Hadoop MapReduce
- Suporte a múltiplas linguagens: Python, Scala, Java, R e SQL
- Processamento unificado: batch, streaming, SQL, machine learning e grafos
- Tolerância a falhas através de RDDs (Resilient Distributed Datasets)
- Integração com diversos sistemas de armazenamento: HDFS, S3, Cassandra, HBase

1.2 História e Evolução

O Apache Spark foi criado por Matei Zaharia em 2009 como um projeto de pesquisa na UC Berkeley. Em 2010, o código foi open-sourced e em 2013 o projeto foi doado a Apache Software Foundation.

Linha do Tempo

Ano	Marco
2009	Criação do projeto na UC Berkeley
2010	Código open-sourced
2013	Doação a Apache Software Foundation
2014	Versão 1.0 lançada
2016	Versão 2.0 com Structured Streaming
2018	Versão 2.4 com suporte a Kubernetes
2020	Versão 3.0 com melhorias de performance
2023	Versão 3.4 com novos recursos de SQL
2024+	Versão 4.0 em desenvolvimento

Tabela 1: Evolução do Apache Spark

1.3 Casos de Uso

O Apache Spark e utilizado em diversos setores e cenários de processamento de dados:

Processamento de Dados em Larga Escala (ETL)

O Spark e amplamente utilizado para pipelines de ETL (Extract, Transform, Load), permitindo a ingestão, transformação e carga de grandes volumes de dados de forma distribuída e eficiente.

Analise de Dados em Tempo Real

Com o Spark Streaming e Structured Streaming, organizações podem processar fluxos de dados em tempo real para analise imediata, detecção de fraudes, monitoramento de sistemas e mais.

Machine Learning

O MLlib fornece algoritmos de machine learning escaláveis para classificação, regressão, clustering, filtragem colaborativa e redução de dimensionalidade.

Processamento de Grafos

O GraphX permite a análise de grafos em escala, útil para análise de redes sociais, detecção de comunidades, caminhos mais curtos e algoritmos de PageRank.

2. Arquitetura do Apache Spark

2.1 Componentes Principais

A arquitetura do Apache Spark e composta por vários componentes que trabalham em conjunto para fornecer processamento distribuído eficiente:

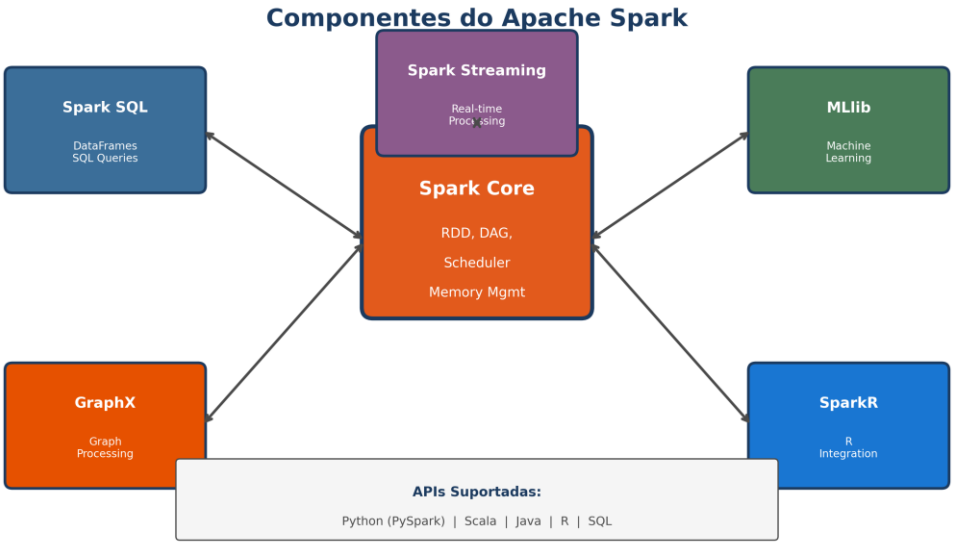

Figura 1: Componentes do Apache Spark

Spark Core

O Spark Core e a base do framework, fornecendo funcionalidades essenciais como gerenciamento de memória, recuperação de falhas, agendamento de tarefas e interação com sistemas de armazenamento. O conceito central e o RDD (Resilient Distributed Dataset).

Spark SQL

Spark SQL fornece suporte para processamento de dados estruturados e semiestruturados. Permite executar consultas SQL e trabalhar com DataFrames e Datasets, oferecendo otimização de consultas através do Catalyst Optimizer.

Spark Streaming

Permite o processamento de fluxos de dados em tempo real. O Structured Streaming fornece uma API unificada para processamento batch e streaming, tratando streams como tabelas em continua atualização.

MLlib

Biblioteca de machine learning escalável que fornece algoritmos comuns de ML, incluindo classificação, regressão, clustering, filtragem colaborativa e redução de dimensionalidade.

GraphX

API para processamento de grafos e computação paralela de grafos, fornecendo algoritmos como PageRank, componentes conectados e caminhos mais curtos.

2.2 Modelo de Execução

O modelo de execução do Spark e baseado em um driver program que coordena a execução de tarefas em um cluster de worker nodes.

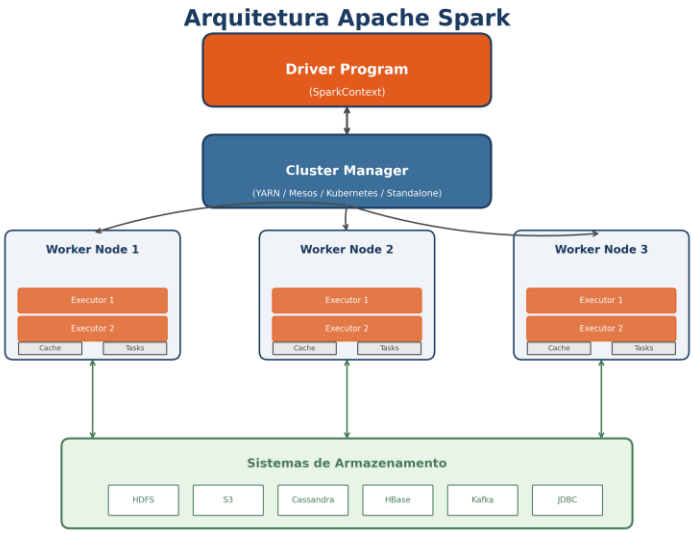

Figura 2: Arquitetura de Execução do Apache Spark

Driver Program

O Driver Program e o processo principal que executa a função main() do aplicativo e cria o SparkContext. Ele e responsável por converter o programa do usuário em tarefas e agendar sua execução nos executors.

Cluster Manager

O Cluster Manager e responsável por alocar recursos entre aplicativos Spark. O Spark suporta vários gerenciadores de cluster:

- Standalone: Gerenciador simples incluído com o Spark
- YARN: Gerenciador de recursos do Hadoop
- Mesos: Gerenciador de cluster genérico
- Kubernetes: Orquestrador de containers

Worker Nodes e Executors

Worker nodes são maquinas no cluster que executam tarefas. Cada aplicativo Spark tem seus próprios executors, que são processos que executam tarefas e armazenam dados em memória ou disco.

3. Instalação e Configuração

3.1 Pré-requisitos

Antes de instalar o Apache Spark, certifique-se de que os seguintes pré-requisitos estejam atendidos:

Requisitos de Hardware

Componente	Requisito
CPU	Multi-core (recomendado 4+ cores)
RAM	Minimo 8GB (recomendado 16GB+)
Disco	20GB+ espaço livre
Rede	Gigabit Ethernet (para clusters)

Tabela 2: Requisitos de Hardware

Requisitos de Software

- Java 8 ou 11 (OpenJDK ou Oracle JDK)
- Python 3.7+ (para PySpark)
- Scala 2.12+ (opcional, para desenvolvimento Scala)
- Hadoop 2.7+ (opcional, para integração com HDFS)

3.2 Instalação Passo a Passo

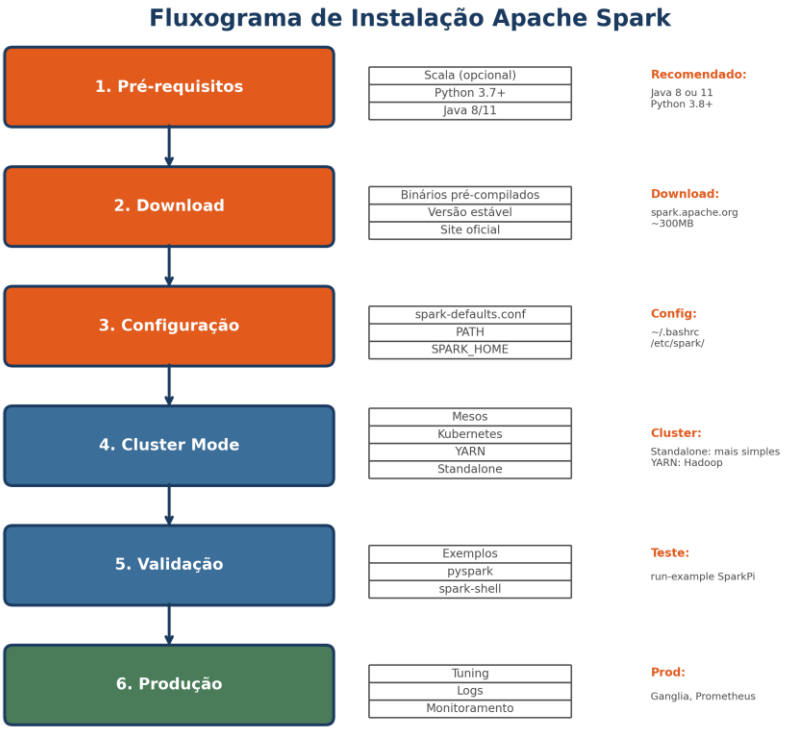

Figura 3: Fluxograma de Instalação do Apache Spark

Passo 1: Instalar Java

Verifique se o Java está instalado:

```
java -version
```

Se não estiver instalado, instale o OpenJDK:

```
# Ubuntu/Debian
sudo apt-get update sudo apt-get install openjdk-11-jdk
# CentOS/RHEL
sudo yum install java-11-openjdk-devel
```

Passo 2: Baixar o Apache Spark

Acesse o site oficial do Apache Spark e baixe a versão mais recente. Escolha a versão pre-compilada para Hadoop:

```
# Download via wget
wget https://archive.apache.org/dist/spark/spark-3.5.0/spark-3.5.0-bin-hadoop3.tgz
```

```
# Extrair
tar -xzf spark-3.5.0-bin-hadoop3.tgz
# Mover para diretório de instalação
sudo mv spark-3.5.0-bin-hadoop3 /opt/spark
```

Passo 3: Configurar Variáveis de Ambiente

Adicione as seguintes variáveis ao arquivo ~/.bashrc ou ~/.profile:

```
export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin export PYSPARK_PYTHON=python3
```

Recarregue as configurações:

```
source ~/.bashrc
```

Passo 4: Verificar Instalação

Teste a instalação executando o shell interativo do Spark:

```
# Spark Shell (Scala)
spark-shell
# PySpark (Python)
pyspark
# Executar exemplo
spark-submit $SPARK_HOME/examples/src/main/python/pi.py 10
```

3.3 Configuração do Cluster

Para configurar um cluster Spark, e necessário configurar o modo de execução e os parâmetros do cluster.

Modo Standalone

O modo Standalone e o mais simples para iniciar um cluster Spark:

```
# Iniciar Master
$SPARK_HOME/sbin/start-master.sh
# Iniciar Workers (em cada no worker)
$SPARK_HOME/sbin/start-worker.sh spark://master:7077
# Verificar interface web
# http://master:8080
```

Configuração spark-defaults.conf

Edite o arquivo $SPARK_HOME/conf/spark-defaults.conf para configurar parâmetros padrao:

```
spark.master                    spark://master:7077
spark.driver.memory             4g
spark.executor.memory           8g
spark.executor.cores            4
spark.sql.adaptive.enabled      true
spark.sql.adaptive.coalescePartitions.enabled  true
```

Explicação de cada parâmetro:

- **spark.master** → Define o endereço do cluster Spark (neste caso, `spark://master:7077`).
- **spark.driver.memory** → Quantidade de memória alocada para o processo *driver* (4 GB).
- **spark.executor.memory** → Memória destinada a cada *executor* (8 GB).
- **spark.executor.cores** → Número de núcleos de CPU por *executor* (4).
- **spark.sql.adaptive.enabled** → Ativa o *Adaptive Query Execution* (otimização dinâmica de planos de execução).
- **spark.sql.adaptive.coalescePartitions.enabled** → Permite reduzir dinamicamente o número de partições para melhorar performance.

Configuração do Ambiente

Crie o arquivo spark-env.sh a partir do template:

```
cp $SPARK_HOME/conf/spark-env.sh.template $SPARK_HOME/conf/spark-env.sh
```

Adicione as configurações necessarias:

export SPARK_MASTER_PORT=7077

export SPARK_WORKER_CORES=4

export SPARK_WORKER_MEMORY=8g

export SPARK_WORKER_INSTANCES=2

Explicação de cada variável:

- **SPARK_MASTER_PORT** → Porta usada pelo processo *master* do Spark (7077).
- **SPARK_WORKER_CORES** → Número de núcleos de CPU disponíveis para cada *worker* (4).
- **SPARK_WORKER_MEMORY** → Quantidade de memória alocada para cada *worker* (8 GB).
- **SPARK_WORKER_INSTANCES** → Número de instâncias de *workers* que serão iniciadas (2).

Essas variáveis normalmente são definidas no ambiente antes de iniciar o Spark, garantindo que os *workers* tenham os recursos corretos.

4. Processamento ETL

O Apache Spark e amplamente utilizado para pipelines de ETL (Extract, Transform, Load), oferecendo alto desempenho e facilidade de desenvolvimento.

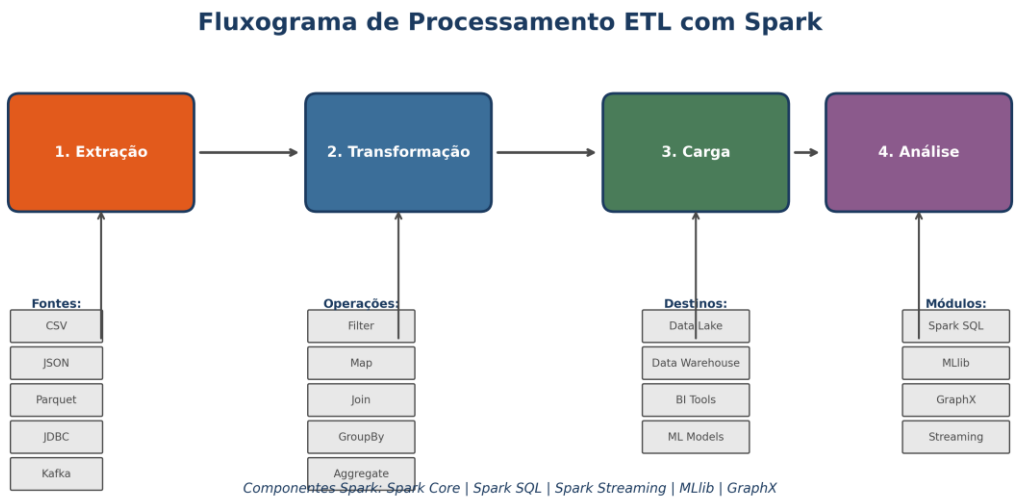

Figura 4: Fluxograma de Processamento ETL

4.1 Extração de Dados

O Spark suporta leitura de diversas fontes de dados. A API DataFrame oferece metodos simples para carregar dados de arquivos CSV, JSON, Parquet, JDBC e outras fontes.

Exemplo de leitura de arquivo CSV:

```
df_csv = spark.read.csv("/path/to/file.csv", header=True, inferSchema=True)
```

Exemplo de leitura de arquivo Parquet:

```
df_parquet = spark.read.parquet("/path/to/file.parquet")
```

Exemplo de leitura via JDBC:

```
df_jdbc = (
  spark.read.format("jdbc")
    .option("url", "jdbc:postgresql://host:port/db")
    .option("dbtable", "table_name")
    .option("user", "username")
    .option("password", "password")
    .load()
)
```

Explicação de cada parâmetro:

- **format("jdbc")** → Define que a fonte de dados é JDBC.
- **url** → String de conexão JDBC para o banco PostgreSQL (`jdbc:postgresql://host:port/db`).
- **dbtable** → Nome da tabela que será lida.
- **user** → Usuário do banco de dados.
- **password** → Senha do banco de dados.
- **load()** → Executa a leitura e carrega os dados em um DataFrame Spark.

Esse bloco pode ser colocado diretamente em um script PySpark para carregar dados de uma tabela PostgreSQL em um DataFrame.

4.2 Transformação de Dados

O Spark oferece uma rica API para transformação de dados. As operações comuns incluem filtragem, seleção, agregação, joins e ordenação.

Exemplos de transformações:

```
# Filtrar dados
df_filtered = df.filter(df.age > 18)

# Selecionar colunas
df_selected = df.select("name", "age", "city")
```

```
# Agregações
from pyspark.sql.functions import avg
df_agg = df.groupBy("city").agg(avg("age").alias("avg_age"))

# Joins
df_joined = df1.join(df2, df1.id == df2.id, "inner")

# Ordenação
df_sorted = df.orderBy(df.age.desc())
```

Explicação de cada comando:

- **df.filter(df.age > 18)** → Filtra apenas registros com idade maior que 18.
- **df.select("name", "age", "city")** → Seleciona apenas as colunas desejadas.
- **groupBy("city").agg(avg("age").alias("avg_age"))** → Agrupa por cidade e calcula a média das idades.
- **join(df2, df1.id == df2.id, "inner")** → Faz um *inner join* entre dois DataFrames usando a coluna `id`.
- **orderBy(df.age.desc())** → Ordena os dados pela idade em ordem decrescente.

Esse conjunto de operações cobre o fluxo básico de manipulação de dados em PySpark: **filtragem, seleção, agregação, junção e ordenação**.

4.3 Carga de Dados

Após as transformações, os dados podem ser salvos em diversos formatos e destinos.

Exemplos de escrita:

```
# Salvar como Parquet (recomendado)
df.write.parquet("/output/path")

# Salvar como CSV
df.write.csv("/output/path", header=True)

# Salvar em banco de dados (PostgreSQL via JDBC)
df.write.format("jdbc") \
    .option("url", "jdbc:postgresql://host:port/db") \
    .option("dbtable", "output_table") \
    .mode("overwrite") \
    .save()

# Particionamento para otimização
df.write.partitionBy("year", "month").parquet("/output/path")
```

Explicação de cada comando:

- **df.write.parquet("/output/path")** → Salva os dados em formato Parquet, que é binário, compacto e eficiente para leitura.
- **df.write.csv("/output/path", header=True)** → Exporta os dados como CSV, incluindo cabeçalho.
- **df.write.format("jdbc")...save()** → Grava os dados diretamente em uma tabela de banco de dados PostgreSQL via JDBC.
- **df.write.partitionBy("year", "month").parquet("/output/path")** → Cria partições por ano e mês ao salvar em Parquet, melhorando performance em consultas filtradas.

Esses exemplos cobrem os principais cenários de saída: **arquivos (Parquet/CSV), banco de dados e otimização com particionamento**.

4.4 Otimização de Performance

Para maximizar a performance de pipelines ETL no Spark:

Explicação de como maximizar a performance de pipelines ETL no Spark:

- **Use formatos colunares como Parquet ou ORC**
  Esses formatos armazenam dados em colunas, permitindo leitura seletiva apenas das colunas necessárias. Isso reduz I/O, melhora compressão e acelera consultas analíticas em grandes volumes de dados.
- **Particione os dados adequadamente para paralelismo**
  O particionamento divide os dados em blocos menores, distribuindo-os entre os *executors*. Isso aumenta o paralelismo e evita gargalos. É importante escolher chaves de partição que tenham boa distribuição (evitar *skew*).
- **Utilize `cache()` ou `persist()` para reutilizar DataFrames**
  Quando um mesmo DataFrame é usado em múltiplas etapas, armazená-lo em memória (ou disco, dependendo da estratégia de persistência) evita recomputações caras. Isso é útil em pipelines com transformações repetidas.
- **Evite *shuffles* desnecessários com operações otimizadas**
  O *shuffle* (redistribuição de dados entre nós) é uma das operações mais custosas no Spark. Usar funções como `mapPartitions`, `reduceByKey` ou `broadcast joins` pode reduzir o número de *shuffles* e melhorar a performance.
- **Habilite o Adaptive Query Execution (AQE)**
  O AQE ajusta dinamicamente o plano de execução durante o runtime, otimizando número de partições, estratégias de *join* e balanceamento de carga. Isso resulta em consultas mais rápidas e eficientes, especialmente em cenários com dados desbalanceados.

Em resumo: **formato eficiente + particionamento inteligente + reutilização de dados + minimização de shuffles + AQE** formam a base para pipelines Spark de alta performance.

5. Machine Learning com MLlib

O MLlib e a biblioteca de machine learning escalável do Apache Spark, projetada para ser fácil de usar, escalável e integrar-se perfeitamente com outros componentes do Spark.

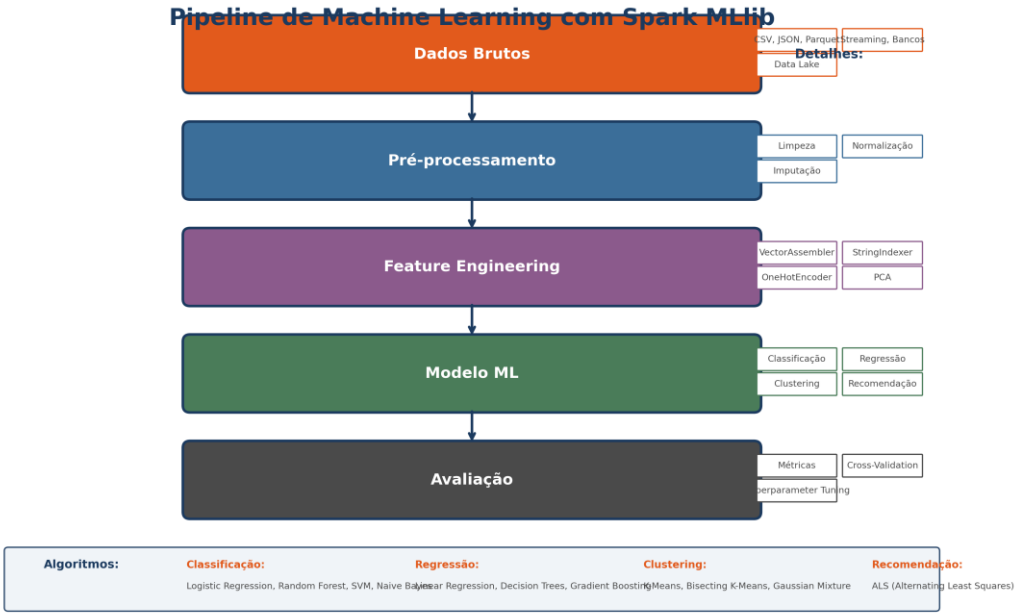

Figura 5: Pipeline de Machine Learning com Spark MLlib

5.1 Tipos de Algoritmos

O MLlib oferece uma ampla variedade de algoritmos de machine learning:

Classificação

- Logistic Regression
- Decision Trees e Random Forests
- Gradient Boosted Trees
- Support Vector Machines (SVM)
- Naive Bayes
- Multilayer Perceptron (Redes Neurais)

Regressão

- Linear Regression
- Decision Tree Regression
- Random Forest Regression
- Gradient Boosted Regression
- Isotonic Regression

Clustering

- K-Means
- Bisecting K-Means
- Gaussian Mixture
- Power Iteration Clustering
- LDA (Latent Dirichlet Allocation)

Filtragem Colaborativa

- ALS (Alternating Least Squares) para recomendação

5.2 Exemplo de Pipeline ML

A seguir, um exemplo completo de pipeline de classificação usando PySpark:

```
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# Carregar dados
data = spark.read.format("libsvm").load("data/sample_libsvm_data.txt")

# Dividir em treino e teste
train, test = data.randomSplit([0.7, 0.3])

# Criar modelo de regressão logística
lr = LogisticRegression(maxIter=10, regParam=0.3)
```

```
# Treinar modelo
model = lr.fit(train)

# Fazer previsões
predictions = model.transform(test)

# Avaliar modelo
evaluator = BinaryClassificationEvaluator()
accuracy = evaluator.evaluate(predictions)
```

Explicação de cada etapa:

- **Importações** → Carrega classes necessárias para pipeline, modelo e avaliação.
- **Leitura dos dados** → Usa formato `libsvm`, comum em datasets de classificação binária.
- **Divisão treino/teste** → Separa 70% para treino e 30% para teste.
- **Criação do modelo** → `LogisticRegression` com 10 iterações e parâmetro de regularização `0.3`.
- **Treinamento** → Ajusta o modelo com os dados de treino.
- **Predições** → Aplica o modelo nos dados de teste.
- **Avaliação** → Mede a performance usando `BinaryClassificationEvaluator` (por padrão, AUC).

Esse fluxo é um exemplo clássico de **pipeline de classificação binária em PySpark**.

5.3 Feature Engineering

O MLlib fornece diversos transformers para feature engineering:

```
from pyspark.ml.feature import VectorAssembler, StringIndexer, OneHotEncoder, StandardScaler

# VectorAssembler - combinar colunas em vetor
assembler = VectorAssembler(
   inputCols=["feature1", "feature2"],
   outputCol="features"
)

# StringIndexer - converter strings em índices
indexer = StringIndexer(
   inputCol="category",
   outputCol="categoryIndex"
)

# OneHotEncoder - codificação one-hot
encoder = OneHotEncoder(
   inputCol="categoryIndex",
   outputCol="categoryVec"
)
```

```
# StandardScaler - normalização
scaler = StandardScaler(
    inputCol="features",
    outputCol="scaledFeatures"
)
```

Explicação de cada etapa:

- **VectorAssembler** → Combina múltiplas colunas numéricas em um único vetor de recursos (`features`), necessário para algoritmos de ML.
- **StringIndexer** → Converte valores categóricos em índices numéricos (ex.: "A", "B", "C" → 0, 1, 2).
- **OneHotEncoder** → Transforma os índices em vetores binários (ex.: índice 2 → `[0,0,1]`), evitando que o modelo interprete ordem numérica inexistente.
- **StandardScaler** → Normaliza os valores numéricos, garantindo que todas as features tenham escala comparável (média 0, variância 1).

Esses componentes são normalmente encadeados em um **Pipeline do Spark ML**, permitindo que o pré-processamento seja aplicado automaticamente antes do treinamento do modelo.

6. Analise de Risco

O Apache Spark e uma ferramenta poderosa para análise de risco em setores financeiros, seguradoras e outras indústrias que requerem avaliação de risco em larga escala.

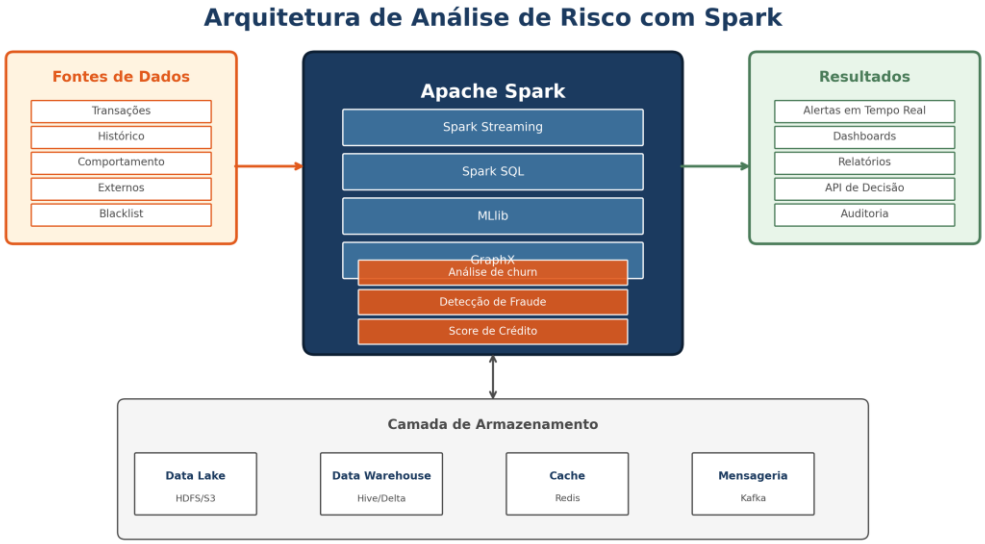

Figura 6: Arquitetura de Análise de Risco com Spark

6.1 Fontes de Dados de Risco

Sistemas de análise de risco tipicamente integram múltiplas fontes de dados:

- Dados de transações financeiras em tempo real
- Histórico de credito e comportamento de pagamento
- Dados comportamentais e de navegação
- Fontes externas: bureaus de credito, listas de vigilância
- Dados de dispositivos e geolocalização

6.2 Modelos de Risco

O Spark MLlib pode ser utilizado para construir diversos modelos de risco:

Score de Credito

Modelos de regressão logística ou gradient boosting para prever a probabilidade de inadimplência:

```
from pyspark.ml.classification import GBTClassifier

# Criar modelo de Gradient Boosting
gbt = GBTClassifier(
   labelCol="default",
   featuresCol="features",
   maxIter=50,
   maxDepth=5
)

# Treinar modelo
model = gbt.fit(training_data)

# Fazer previsões
predictions = model.transform(test_data)
```

Explicação de cada etapa:

- **GBTClassifier** → Algoritmo de classificação baseado em *Gradient Boosted Trees*, eficiente para problemas binários e multiclasse.
- **labelCol="default"** → Define a coluna alvo (variável dependente).
- **featuresCol="features"** → Define a coluna que contém o vetor de atributos.
- **maxIter=50** → Número máximo de iterações (quantidade de árvores a serem construídas).

- **maxDepth=5** → Profundidade máxima de cada árvore, controlando complexidade e risco de *overfitting*.
- **fit(training_data)** → Treina o modelo com os dados de treino.
- **transform(test_data)** → Aplica o modelo nos dados de teste para gerar previsões.

Esse fluxo é um exemplo clássico de uso de **Gradient Boosting em PySpark ML**.

Detecção de Fraude

Algoritmos de detecção de anomalias e classificação para identificar transações suspeitas:

```
from pyspark.ml.clustering import KMeans
from pyspark.ml.feature import StandardScaler
from pyspark.sql.functions import col

# Normalizar features
scaler = StandardScaler(
    inputCol="features",
    outputCol="scaledFeatures",
    withStd=True,
    withMean=False
)

scaled_data = scaler.fit(data).transform(data)

# Clustering para detecção de anomalias
kmeans = KMeans(
    k=5,
    featuresCol="scaledFeatures",
    predictionCol="cluster"
)

model = kmeans.fit(scaled_data)

# Fazer previsões (atribuir cada ponto a um cluster)
predictions = model.transform(scaled_data)

# Identificar outliers baseado na distância ao centroide
# Calcula a distância de cada ponto ao centroide do cluster
centers = model.clusterCenters()

predictions = predictions.withColumn(
    "distanceToCenter",
    col("scaledFeatures").apply(lambda v: float(v.squared_distance(centers[col("cluster")]))))
)
```

Explicação de cada etapa:

- **StandardScaler** → Normaliza os dados para que todas as features tenham escala comparável (evita que uma variável com valores grandes domine o clustering).
- **KMeans(k=5)** → Cria 5 clusters, agrupando os dados em torno de centroides.
- **model.fit(scaled_data)** → Treina o modelo com os dados normalizados.
- **model.transform(scaled_data)** → Atribui cada ponto ao cluster mais próximo.
- **distanceToCenter** → Mede a distância de cada ponto ao centroide do cluster. Valores muito altos podem indicar **anomalias/outliers**.

Esse fluxo é um exemplo de como usar **KMeans para detecção de anomalias** em PySpark.

6.3 Processamento em Tempo Real

Para análise de risco em tempo real, o Structured Streaming e essencial:

```
# Ler stream de Kafka
stream_df = (
   spark.readStream
      .format("kafka")
      .option("kafka.bootstrap.servers", "host:port")
      .option("subscribe", "transactions")
      .load()
)

# Aplicar modelo de ML (assumindo que parsed_df já foi transformado em features)
predictions = model.transform(parsed_df)

# Filtrar alertas de alto risco
alerts = predictions.filter(predictions.prediction == 1)

# Escrever alertas em banco de dados
query = (
   alerts.writeStream
      .outputMode("append")
      .format("jdbc")
      .option("url", "jdbc:postgresql://host:port/db")
      .option("dbtable", "fraud_alerts")
      .option("user", "username")
      .option("password", "password")
      .start()
)
```

Explicação de cada etapa:

- **spark.readStream.format("kafka")** → Conecta ao Kafka para consumir mensagens em tempo real do tópico `transactions`.

- **model.transform(parsed_df)** → Aplica o modelo de ML previamente treinado sobre os dados processados do stream.
- **filter(predictions.prediction == 1)** → Seleciona apenas os registros classificados como alto risco (fraude).
- **writeStream.format("jdbc")** → Persiste os alertas em uma tabela de banco de dados PostgreSQL (`fraud_alerts`), em modo *append* (inserindo novos registros continuamente).

Esse fluxo é um exemplo de **pipeline de detecção de fraude em tempo real** usando Spark Structured Streaming + Kafka + ML + JDBC.

6.4 Governança e Compliance

Sistemas de análise de risco devem atender requisitos regulatórios:

- Rastreabilidade completa de decisões de modelo
- Versionamento de modelos e experimentos (MLflow)
- Auditoria de dados e transformações
- Explicabilidade de predicoes (SHAP, LIME)
- Controle de acesso e segurança de dados

7. Tecnologias Emergentes

O ecossistema do Apache Spark continua evoluindo com novas tecnologias e integrações que expandem suas capacidades.

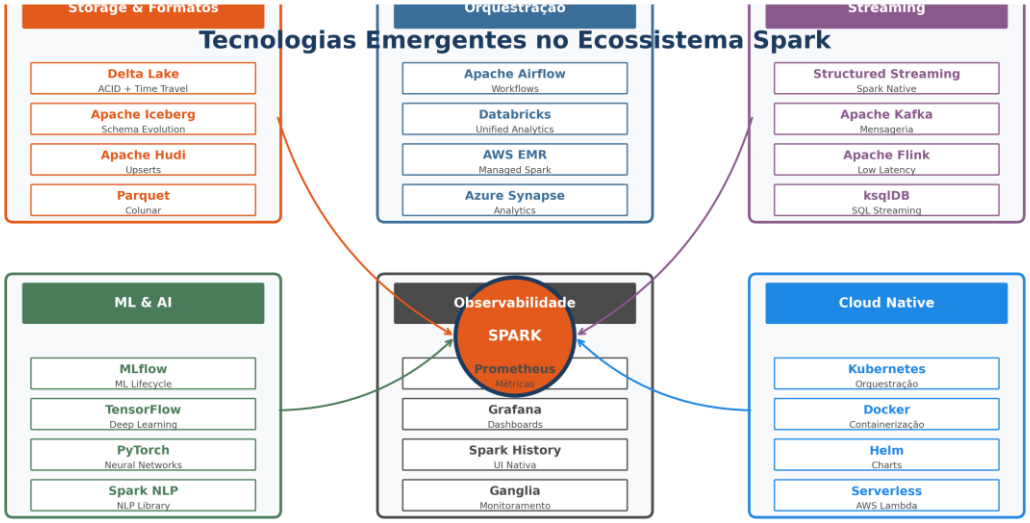

Figura 7: Tecnologias Emergentes no Ecossistema Spark

7.1 Lakehouse Architecture

A arquitetura Lakehouse combina a flexibilidade de data lakes com a confiabilidade de data warehouses:

- Delta Lake: Camada de storage que adiciona transacoes ACID ao data lake
- Apache Iceberg: Formatos de tabela para grandes datasets analiticos
- Apache Hudi: Framework para ingestão de dados em grande escala
- Time Travel: Capacidade de consultar dados em pontos especificos no tempo
- Schema Evolution: Evolução do esquema sem reescrita de dados

7.2 Kubernetes e Cloud Native

A integração com Kubernetes permite orquestração moderna de clusters Spark:

```
spark-submit \
  --master k8s://https://<k8s-api-server> \
  --deploy-mode cluster \
  --name spark-pi \
  --class org.apache.spark.examples.SparkPi \
  --conf spark.executor.instances=3 \
  --conf spark.kubernetes.container.image=spark:3.5.0 \
  local:///path/to/examples.jar
```

Explicação de cada parâmetro:

- **--master k8s://https://** → Define o cluster Kubernetes como ambiente de execução, apontando para o servidor da API.
- **--deploy-mode cluster** → Executa o job diretamente no cluster (não no cliente).
- **--name spark-pi** → Nome do job Spark (nesse caso, `spark-pi`).
- **--class org.apache.spark.examples.SparkPi** → Classe principal a ser executada dentro do JAR.
- **--conf spark.executor.instances=3** → Número de executores que serão criados no cluster.
- **--conf spark.kubernetes.container.image=spark:3.5.0** → Imagem Docker usada para os pods Spark no Kubernetes.
- **local:///path/to/examples.jar** → Caminho para o JAR de exemplo, acessível dentro do container.

Esse comando é um exemplo clássico de como rodar Spark sobre Kubernetes usando `spark-submit`.

7.3 AI e Deep Learning

A integração do Spark com frameworks de deep learning está se expandindo:

- Horovod: Framework distribuído para treinamento de deep learning
- TensorFlow On Spark: Executa TensorFlow em clusters Spark
- Deep Learning Pipelines: APIs de alto nível para DL no Spark
- Spark NLP: Biblioteca NLP de produção com suporte a transformers

7.4 Streaming Avançado

Novas capacidades para processamento de streams:

- **Continuous Processing**
  Permite que o Spark Structured Streaming opere com **latência de milissegundos**, em vez de segundos. Isso é possível porque o motor de execução processa os dados de forma contínua, sem esperar por micro-batches. É ideal para aplicações que exigem respostas quase em tempo real, como detecção de fraudes ou monitoramento de sensores.
- **Watermarking**
  Define um limite de tempo para considerar dados atrasados (*late data*). Isso evita que eventos muito antigos continuem sendo processados indefinidamente, controlando memória e garantindo que janelas de agregação sejam finalizadas. Exemplo: `withWatermark("timestamp", "10 minutes")` permite aceitar dados atrasados até 10 minutos.
- **Windowed Operations**
  Permitem realizar **agregações em janelas temporais** (por exemplo, somar vendas a cada 5 minutos). Isso é essencial para análises de séries temporais e métricas contínuas. Pode ser usado em conjunto com watermarking para lidar com dados fora de ordem.
- **Exactly-once Semantics**
  Garante que cada evento seja processado **uma única vez**, sem duplicação, mesmo em cenários de falhas ou reinícios. Essa característica é crítica para aplicações financeiras ou de auditoria, onde duplicar ou perder eventos pode causar inconsistências graves.

Em resumo: essas capacidades tornam o **Spark Structured Streaming** muito mais robusto e adequado para cenários de **tempo real**, combinando baixa latência, tolerância a atrasos, agregações temporais e confiabilidade no processamento.

7.5 Observabilidade

Ferramentas modernas para monitoramento de clusters Spark:

- Prometheus + Grafana: Coleta e visualizacao de metricas
- Spark History Server: Analise de jobs completados
- Ganglia: Monitoramento de performance do cluster
- Spark Listener API: Customizacao de metricas e eventos

8. Conclusão

O Apache Spark consolidou-se como uma das principais tecnologias para processamento de dados em larga escala, oferecendo um framework unificado que atende desde cargas de trabalho batch tradicionais até processamento em tempo real, machine learning e analise de grafos.

8.1 Resumo dos Benefícios

- Alta performance através de processamento em memoria
- APIs intuitivas em múltiplas linguagens de programação
- Ecossistema rico com integrações para diversas fontes de dados
- Escalabilidade horizontal para processamento de petabytes
- Tolerância a falhas integrada
- Comunidade ativa e desenvolvimento continuo

8.2 Próximos Passos

Para aprofundar seus conhecimentos em Apache Spark:

0. Explore a documentacao oficial em spark.apache.org
1. Pratique com exemplos e datasets públicos
2. Participe da comunidade através de foruns e meetups
3. Considere certificações como Databricks Certified Developer
4. Acompanhe as novidades nas releases e projetos relacionados

8.3 Recursos Adicionais

Recurso	Link/Editora
Documentação Oficial	spark.apache.org/docs/latest
GitHub	github.com/apache/spark

Recurso	Link/Editora
Databricks	databricks.com/spark
Stack Overflow	stackoverflow.com/questions/tagged/apache-spark
Livro: Learning Spark	O'Reilly Media
Livro: Spark: The Definitive Guide	O'Reilly Media

Tabela 3: Recursos Adicionais para Aprendizado

Apache Spark

Guia Técnico Completo

Processamento Distribuído de Dados em Larga Escala

Documentação Técnica - Março 2026

Professor Benir Falcão Pistille

Versão – 1.0